1. MPTCP vs MP-QUIC: A Comparative Study

Theoretical Foundations, Architectural Deconstruction & Literature Review

Course: CS 331 — Computer Networks | Team ID: T007

Project ID: [Pending Allocation / Instructor Confirmation]

Team Members & Division of Responsibilities:

- Kaushal Rathi (24110290): QUIC & MP-QUIC Design; Comparative Analysis Lead
- Krina Paghadar (24110169): MPTCP Architecture, Linux Kernel Status & Adoption Lead
- Pratik Kumar Dhakar (24110277): Coupled Congestion Control, Scheduling & Methodology Lead
- Swamini More (24110211): Published Research Synthesis & Application Implications Lead

GitHub Repository: https://github.com/Krina-Paghadar/CS331-T007-MPTCP-vs-MPQUIC

Speaker Note / Primary Source: *Slide 1 Speaker Notes: Welcome Sir and peers. This is Team T007 presenting our CS 331 Computer Networks course project: 'MPTCP vs MP-QUIC: A Comparative Study of Multipath Transport Protocols'. Our team comprises Kaushal, Krina, Pratik, and Swamini. This presentation strictly covers a first-principles theoretical, architectural, and published literature study comparing MPTCP v1 (RFC 8684) and MP-QUIC (draft-ietf-quic-multipath-21). Source: Project Charter and Team Allocation.*

2. Why Multipath: Multihoming vs. Transport Reality

The Physical Reality of Multiple Links vs. The Single-Path Transport Bottleneck

Ubiquitous Hardware Multihoming: Modern smartphones, edge nodes, and vehicles feature concurrent Wi-Fi, 5G/LTE, and Bluetooth interfaces. Datacenter Clos fabrics provide dense parallel paths.

The Legacy Transport Bottleneck: Traditional TCP (RFC 793 / RFC 9293) and baseline QUIC (RFC 9000) bind a connection immutably to a single IP 4-tuple. Changing an IP address breaks the connection.

Multipath Transport Goals (RFC 6182):

1. Capacity Aggregation: Combine bandwidth across multiple physical interfaces for high throughput.

2. Resilience & Seamless Handover: Maintain active connections uninterrupted when an interface fails.

3. Latency Optimization: Steer latency-critical packets onto the lowest-delay path.

The Practical Reality: Despite 15+ years of research, multipath transport remains rare on the open Internet due to middlebox hostility, asymmetric delay traps, and operating system API constraints.

Speaker Note / Primary Source: *Slide 2 Speaker Notes: Why does multipath transport matter, yet why is it rarely used? While our devices have multiple radios, TCP and single-path QUIC freeze connections to one 4-tuple. Multipath transport solves this by pooling resources across links. However, middleboxes that strip options, asymmetric RTTs that cause reordering, and UDP filtering have held back widespread deployment. Primary sources: RFC 6182 Section 1, RFC 8041, and Honda et al. (IMC 2011).*

3. MPTCP Architecture: Two Sequence Spaces & Subflows

How MPTCP Preserves Legacy TCP Compatibility Across Multiple Network Paths

Protocol Layering (RFC 8684): Sits inside the OS kernel transport layer. Exposes a standard POSIX socket (SOCK_STREAM) to applications while multiplexing data over multiple TCP subflows.

Dual-Level Sequence Numbering (RFC 8684 §3.3):

• Subflow Sequence Number (SSN, 32-bit): Governs standard TCP hop-by-hop reliability and window flow control per subflow. Middleboxes observe standard TCP byte streams.

• Data Sequence Number (DSN, 64-bit): Enforces connection-wide ordered byte delivery to the application.

Data Sequence Signal (DSS) Option (RFC 8684 §3.3): Carries explicit byte-interval mappings (SSN to DSN), a 64-bit Data ACK, and an optional DSS checksum to detect middlebox payload rewriting.

Fallback Protection (RFC 8684 §3.6): If an on-path proxy strips TCP options or mangles payload data, MPTCP triggers MP_FAIL and gracefully falls back to standard single-path TCP with an infinite mapping.

Speaker Note / Primary Source: *Slide 3 Speaker Notes: Here we deconstruct MPTCP's core architecture from RFC 8684 Section 3.3. To pass through middleboxes without being dropped, every subflow must look like ordinary TCP. MPTCP achieves this using two sequence spaces: a 32-bit Subflow Sequence Number for hop-by-hop TCP semantics, and a 64-bit Data Sequence Number for connection-wide reassembly. The DSS option carries the explicit mapping and Data ACK. Primary source: RFC 8684 Sections 1.1, 3.3, and 3.6.*

4. MPTCP Handshake & Subflow Joining

Cryptographic Negotiation, Stateless Servers, and Dynamic Subflow Management

Primary Handshake (MP_CAPABLE, RFC 8684 §3.1):

• SYN, SYN/ACK exchange 64-bit sender keys (Key-A, Key-B).

• MPTCP v1 Improvement: The initiator echoes both keys in the 3rd ACK, allowing the server to operate statelessly (SYN cookies) without pre-allocating connection state.

• Tokens & Initial DSNs: Derived deterministically via SHA-256 hashes of the exchanged keys.

Subflow Addition (MP_JOIN, RFC 8684 §3.2): Client initiates on a new 4-tuple; authenticates via HMAC over the receiver's token and random nonces to prevent off-path connection hijacking.

Path Signaling: ADD_ADDR and REMOVE_ADDR announce alternative IP addresses (authenticated via HMAC in v1); MP_PRIO signals whether a subflow is in ACTIVE or BACKUP mode.

Speaker Note / Primary Source: *Slide 4 Speaker Notes: Walking through the MPTCP handshake: In MPTCP v1, MP_CAPABLE exchanges 64-bit keys. Unlike v0 (RFC 6824), v1 echoes both keys in the 3rd ACK, allowing servers to remain completely stateless using SYN cookies. When adding subflows, MP_JOIN uses the receiver's token—a truncated SHA-256 hash—and an HMAC to prevent off-path hijacking. Primary source: RFC 8684 Sections 3.1, 3.2, 3.4, and RFC 6181.*

5. Coupled Congestion Control & Fairness

Resource Pooling and The 'Do No Harm' Principle at Shared Bottlenecks

The Shared Bottleneck Hazard: If two subflows of the same connection run independent AIMD (e.g., standard CUBIC) across a shared physical bottleneck, they claim twice their fair share of bandwidth.

RFC 6356 Design Goals (LIA - Linked Increases Algorithm):

1. Improve Throughput: A multipath flow should perform at least as well as the best single path.

2. Do No Harm: A multipath flow should take no more capacity than a single-path TCP flow on any shared link.

3. Balance Congestion: Shift traffic dynamically away from congested paths toward uncongested paths.

LIA Mathematical Coupling: Increase parameter α couples subflow cwnd expansion inversely proportional to total throughput and RTT: cwnd_r += min(α * bytes_acked * MSS_r / cwnd_total, bytes_acked * MSS_r / cwnd_r).

Advanced Coupled Algorithms: OLIA fixes LIA's Pareto-suboptimality; BALIA balances friendliness and responsiveness.

Speaker Note / Primary Source: *Slide 5 Speaker Notes: Slide 5 addresses congestion control theory. Running uncoupled TCP on two subflows across a shared bottleneck steals unfair capacity from single-path flows. RFC 6356 defines Coupled Congestion Control (LIA). LIA couples window growth across subflows, shifting traffic away from congested links while guaranteeing the connection claims no more capacity than a single-path flow would. Primary sources: RFC 6356, Wischik et al. (NSDI 2011), and Khalili et al. (OLIA).*

6. QUIC and MP-QUIC Architecture (draft-21)

Encrypted, User-Space Multipath Transport Over UDP with Independent Packet Spaces

Clean-Slate User-Space Protocol (RFC 9000; draft-ietf-quic-multipath-21): Operates over UDP; all transport metadata, stream frames, and ACK ranges are authenticated and encrypted via TLS 1.3.

Explicit Path Identification (draft-21 §3): Paths are identified by 32-bit Path IDs (negotiated via initial_max_path_id) and distinct Connection IDs (CIDs) per path, decoupling transport from IP 4-tuples.

Independent Packet Number Spaces (draft-21 §4):

• Each path maintains its own monotonically increasing Packet Number (PN) space starting at 0.

• Zero Retransmission Ambiguity: Lost stream data is retransmitted in a new packet with a new PN.

Dedicated Control Frames (draft-21 §5):

• PATH_ACK: Acknowledges packets for a specific path ID; may travel over faster alternate paths.

• PATH_STATUS: Explicitly signals AVAILABLE or BACKUP status with monotonic sequence numbers.

• PATH_ABANDON: Cleanly terminates an impaired path; context maintained for 3 * PTO.

Speaker Note / Primary Source: *Slide 6 Speaker Notes: Turning to MP-QUIC, specified in draft-ietf-quic-multipath-21: Operating in user space over UDP with TLS 1.3 encryption, MP-QUIC completely eliminates middlebox interference. Crucially, draft-21 mandates one independent packet number space per path. Because packet numbers never repeat upon retransmission, retransmission ambiguity is completely eliminated, and PATH_ACK frames can return over faster paths without distorting RTT estimates. Primary sources: RFC 9000, RFC 9002, and draft-ietf-quic-multipath-21 Sections 3–6.*

7. Side-by-Side Architectural Comparison

Direct Comparison Across Protocol Layering, Mechanics, and Failure Handling

Layering & Scope: MPTCP v1 is in-kernel over TCP (SOCK_STREAM); MP-QUIC is user-space over UDP (Streams).

Handshake Latency: MPTCP requires 1 RTT for initial subflow + 1 RTT for MP_JOIN; MP-QUIC combines transport & TLS 1.3 handshake (1 RTT / 0-RTT setup).

Sequence Numbering: MPTCP uses 32-bit SSN + 64-bit DSN via DSS option; MP-QUIC uses independent per-path PN spaces + per-stream byte offsets.

Head-of-Line (HOL) Blocking: MPTCP enforces connection-wide ordering (loss on one subflow blocks all subflows); MP-QUIC restricts HOL blocking to within individual streams.

Middlebox Traversal: MPTCP is vulnerable to option stripping and sequence rewriting; MP-QUIC is immune to option stripping (encrypted headers) but vulnerable to UDP throttling/blocking.

Loss Recovery: MPTCP relies on subflow TCP dupACKs/RTO; MP-QUIC uses dedicated PATH_ACK and Probe Timeouts (PTO).

Speaker Note / Primary Source: *Slide 7 Speaker Notes: This slide summarizes our comparative matrix. The fundamental difference lies in their design goals: MPTCP sacrificed flexibility for backwards compatibility, leaving it constrained by 40-byte TCP options and connection-wide HOL blocking. MP-QUIC started clean-slate, giving it stream isolation and per-path packet number spaces, at the cost of requiring user-space application integration and dealing with firewall UDP filtering. Primary source: RFC 8684 vs. draft-ietf-quic-multipath-21.*

8. Implementation Landscape Today

Mainline Linux Kernel, Academic Forks, Commercial Uses, and Standardization Status

Linux Kernel Upstream (v5.6+): Merged into mainline Linux in March 2020. Speaks MPTCP v1 (RFC 8684) exclusively. Features netlink path management (mptcpd) and eBPF schedulers (Linux 6.6+).

The Out-of-Tree Fork Trap (v0.96): Pre-5.6 out-of-tree kernel fork implemented MPTCP v0 (RFC 6824). Historical literature (2011–2019) evaluated this fork; their numbers are not comparable to upstream Linux today.

Commercial Deployments: Apple iOS has deployed MPTCP since iOS 7 (2013) for Siri, Maps, and Apple Music (operating strictly in backup/failover mode); 3GPP ATSSS standardized MPTCP proxies for 5G hybrid access.

MP-QUIC Implementations: Currently progressing through IETF standardization (draft-ietf-quic-multipath-21). picoquic provides the primary reference implementation. The 2017 quic-go fork is obsolete; mainline quic-go does not support multipath.

Speaker Note / Primary Source: *Slide 8 Speaker Notes: Examining implementations today: Linux has supported MPTCP v1 in mainline since kernel 5.6. An important academic fact: older papers evaluated the out-of-tree v0.96 fork speaking MPTCP v0, so their numbers cannot be directly compared to modern upstream Linux. For MP-QUIC, draft-21 is in the RFC Editor queue; picoquic is actively maintained, while mainline quic-go still lacks multipath. Primary sources: Linux Kernel documentation, picoquic README, and RFC 8041.*

9. Why MPTCP is Not Widely Adopted

Middlebox Interference, Head-of-Line Blocking, and Operational Inertia

Middlebox Hostility (Honda et al., IMC 2011):

• Firewalls and NATs routinely strip unknown TCP options, killing MP_CAPABLE or MP_JOIN.

• Transparent proxies rewrite sequence numbers, corrupting DSS byte-interval mappings.

The Asymmetric RTT Penalty (Paasch et al., 2014): When paths have divergent RTTs (e.g., 10ms Wi-Fi vs. 100ms LTE), packets sent on the slow path delay application delivery. Out-of-order packets fill the connection receive buffer, causing a zero-window stall that collapses fast-path throughput.

Application & OS Indifference: Standard POSIX sockets lack multi-interface semantics; mobile OSes restrict multipath to closed first-party services to prevent mobile data plan exhaustion.

QUIC UDP Drop Reality: Enterprise firewalls routinely drop or rate-limit UDP traffic on non-53 ports (5–10% of networks), forcing fallbacks to TCP.

Speaker Note / Primary Source: *Slide 9 Speaker Notes: Why is MPTCP not ubiquitous? First, middleboxes: IMC 2011 showed widespread stripping of unknown TCP options. Second, the asymmetric RTT penalty: as Paasch et al. proved, when a 10ms and 100ms path are combined with MinRTT, fast-path packets fill the receive buffer waiting for slow-path bytes, stalling the connection. Third, mobile OSes avoid running cellular in aggregation mode to save battery and data caps. Primary sources: Honda et al. (IMC 2011), Paasch et al. (2014), and arXiv:2106.07351.*

10. Published Empirical Findings (With Setups)

Synthesis of Peer-Reviewed Testbed Evaluations Under Original Documented Conditions

Symmetric Path Aggregation (CoNEXT 2017, Mininet/NetEm): Both MPTCP and MP-QUIC achieve 90–96% capacity aggregation under symmetric clean links (dual 20ms RTT, 0% loss) using MinRTT scheduling.

Loss Asymmetry & Stream Isolation (CoNEXT 2017 §4.2):

• Under 2% packet loss on one path, MPTCP suffers global HOL blocking: all subflows stall.

• MP-QUIC achieves up to 3x higher throughput for multiplexed web objects: loss on one path blocks only the affected stream, allowing concurrent streams to proceed unimpeded.

Mobile Handover & Failover (MultipathTester 2019, Android LTE/Wi-Fi):

• MP-QUIC delivers seamless link migration during physical movement across Wi-Fi and LTE.

• MPTCP experiences multi-second stalls during abrupt Wi-Fi drops due to exponential RTO backoff delays before reinjecting stranded packets onto cellular.

Caveat: CoNEXT 2017 evaluated pre-IETF Google QUIC; modern draft-21 per-path PN spaces yield cleaner loss recovery.

Speaker Note / Primary Source: *Slide 10 Speaker Notes: Synthesizing published empirical literature under original setups: In CoNEXT 2017 (Mininet), symmetric paths aggregate well for both protocols. But when 2% packet loss is introduced on one path, MP-QUIC achieves 3x higher throughput because QUIC confines HOL blocking to individual streams, whereas MPTCP stalls the entire connection. In MultipathTester (2019, Android), MP-QUIC achieved faster failover because MPTCP subflows suffered exponential RTO backoff when Wi-Fi dropped. Primary sources: CoNEXT 2017 Section 4 and MultipathTester 2019 Section 3.*

11. Project Scope, Methodology & Blockers

Transparent Academic Honesty: What We Did, What We Did Not Do, and Why

Strict Scope Definition: This is an in-depth theory, specification, and published literature study.

What the Team Did NOT Do (Non-Negotiable Boundaries):

• Authored no transport protocol code or kernel modules.

• Configured no live physical or emulated multipath testbed (no Mininet, NetEm, or iperf3 runs).

• Collected no live measurement numbers and fabricated no synthetic benchmark plots.

Technical Environment Blockers Encountered:

1. WSL2 Kernel: Stock Windows Subsystem for Linux kernel lacks MPTCP compilation (CONFIG_MPTCP absent).

2. Multipass VM: Canonical Multipass hypervisor drivers failed to launch Ubuntu VMs on Windows.

3. Principled Decision: The team chose not to troubleshoot hypervisor drivers and instead focused on a rigorous, defensible analytical deconstruction of official RFCs, IETF drafts, and peer-reviewed papers.

Foundational Curriculum: Developed a 323-question study plan and executed 4-session teach-backs.

Speaker Note / Primary Source: *Slide 11 Speaker Notes: Slide 11 provides our transparent academic scope. We state plainly: this is a theory and literature study only. We wrote no code and ran no experiments. When we attempted to set up a local Linux testbed, WSL2 lacked MPTCP in its stock kernel and Multipass hypervisor failed on Windows. Rather than debugging hypervisors, we refocused entirely on primary specifications (RFC 8684, draft-21) and a 323-question cross-training curriculum. Primary source: code/README.md and mptcp-mpquic-understanding-plan.md.*

12. Application Recommendations & Conclusion

Matching Transport Strengths to Application Workloads: Video, Bulk, and Real-Time

Adaptive Video Streaming (DASH / HLS): Recommend MP-QUIC in Backup/Handover Mode (or BLEST scheduler). QUIC's per-stream byte offsets prevent video segment download stalls when cellular experiences jitter.

Bulk File Transfer: Recommend MPTCP v1 for legacy TCP server deployments (benefits from in-kernel zero-copy and NIC hardware offloads like TSO/GRO); recommend MP-QUIC for lossy wireless environments.

Real-Time Voice (VoIP): Recommend MP-QUIC with unreliable DATAGRAM frames (RFC 9221). Multipath reordering destroys jitter buffers in TCP; DATAGRAM frames allow path redundancy without unneeded retransmission delays.

Summary Conclusion: MPTCP v1 prioritizes backwards compatibility at the cost of middlebox vulnerability and global HOL blocking. MP-QUIC provides a clean-slate, encrypted, stream-isolated architecture whose widespread deployment awaits finalized RFC publication and OS socket standardization.

Speaker Note / Primary Source: *Slide 12 Speaker Notes: To conclude with application recommendations: For adaptive video, MP-QUIC in backup mode is superior because stream isolation prevents rebuffering stalls. For bulk transfers, MPTCP v1 remains highly efficient for legacy servers due to in-kernel zero-copy offloads, but MP-QUIC wins over lossy links. For VoIP, MP-QUIC with DATAGRAM frames (RFC 9221) is ideal, avoiding TCP retransmission delays. In short: MPTCP prioritized backwards compatibility; MP-QUIC prioritizes deployability and stream isolation. Primary sources: RFC 9221, draft-21, CoNEXT 2017, and RFC 8041.*